Cost-Sensitive Ensemble Machine Learning for SIEM Alert Triage: Reducing Alert Fatigue Through Stacking Classification

Yogita
Student ID: x00000000
National College of Ireland
Department of AI/ML
Graduate Research Project

***Abstract*—Security Information and Event Management (SIEM) systems generate an overwhelming volume of alerts, the vast majority of which are false positives. This alert fatigue leads analysts to miss genuine threats and degrades the effectiveness of Security Operations Centers (SOCs). We propose a cost-sensitive stacking ensemble that combines Random Forest, XGBoost, and Isolation Forest anomaly scores with a Logistic Regression meta-learner to perform three-class alert triage: True Positive, False Positive, and Indeterminate. Our cost function penalises missed true positives (false negatives) at $10\times$ the cost of false positive escalations, directly encoding the asymmetric risk of security operations. Evaluated on synthetic datasets modelled after CIC-IDS2017, NSL-KDD, and UNSW-NB15, the stacking ensemble achieves strong $F_{\beta=2}$ scores while significantly reducing alert volume. SHAP-based explanations identify the most influential alert features, providing interpretable decision support for analysts. The system demonstrates that intelligent ML-driven triage can substantially reduce analyst workload without compromising threat detection.**

***Index Terms*—SIEM, alert triage, cost-sensitive learning, ensemble methods, stacking classifier, alert fatigue, XGBoost, Random Forest, Isolation Forest, SHAP**

I. Introduction

Modern enterprises rely on Security Information and Event Management (SIEM) systems to aggregate and correlate security events from firewalls, intrusion detection systems (IDS), endpoints, and cloud infrastructure [1]. However, the volume of alerts generated daily—often tens of thousands—far exceeds the capacity of human analysts, with false positive rates commonly reported above 80% [2], [3]. This phenomenon, widely known as *alert fatigue*, causes analysts to overlook genuine threats, increases mean time to respond (MTTR), and contributes to analyst burnout [4].

Recent work has explored machine learning approaches to alert triage, including graph-based prioritisation [5], context-aware alert prioritisation [6], and empirical measurement of alert accuracy in production SOCs [7]. While promising, many existing methods treat the problem as binary classification (benign vs. malicious) and fail to account for the *asymmetric costs* inherent in security operations: missing a true attack is far more damaging than investigating a false alarm [8].

In this work, we address these limitations through three contributions:

1) **Three-class triage**: We formulate alert triage as a three-class problem (True Positive, False Positive, Indeterminate), reflecting real-world SOC workflows where uncertain alerts require additional investigation rather than outright dismissal.
2) **Cost-sensitive stacking ensemble**: We combine Random Forest with class-weighted losses, XGBoost with sample-level cost weights, and Isolation Forest anomaly scores within a stacking architecture. The meta-learner (Logistic Regression) fuses predictions to maximise $F_{\beta=2}$, which prioritises recall to minimise missed threats.
3) **Explainable triage**: We apply SHAP (SHapley Additive exPlanations) to identify the features most influential in classifying an alert as a true positive, providing actionable insight for analysts.

II. Related Work

A. SIEM Architecture and Alert Management

Bhatt et al. [9] provided one of the earliest systematic analyses of the operational role of SIEM systems, documenting how log aggregation, correlation rules, and alert escalation workflows interact in enterprise environments. Their work highlighted that SIEM effectiveness is fundamentally limited by the quality of alert triage, a finding that motivates the ML-driven approach we pursue here. However, their analysis predates modern ensemble learning methods and does not address the cost-asymmetry inherent in security triage.

Ban et al. [4] proposed an AI-assisted SIEM framework for breaking alert fatigue through effective incident response using random forests and gradient boosting. While their results demonstrate the viability of tree-based ensembles for alert classification, they employ binary triage (malicious vs. benign) and do not incorporate cost-sensitive loss functions, leaving a gap for high-stakes environments where missed true positives carry disproportionate consequences. Ghadermazi et al. [3] developed a machine learning and optimisation framework

for efficient alert management in cybersecurity operations centres, combining clustering with supervised classification. Their optimisation-based approach is complementary to our cost-sensitive formulation, though it does not exploit stacking ensembles or provide per-alert explainability. Khayat et al. [2] provided a systematic literature review on empowering SOCs with AI and ML, identifying ensemble methods and cost-sensitive learning as key research gaps that remain underexplored in the literature.

B. Alert Fatigue and SOC Analyst Studies

Alahmadi et al. [10] conducted a qualitative study of SOC analysts' perspectives on security alarms, finding that analysts routinely face false positive rates exceeding 99% and develop coping strategies—such as ignoring entire alert categories—that can silently degrade detection capability. Their findings provide strong empirical motivation for automated triage systems that can reliably filter false positives without suppressing genuine threats. Tariq et al. [1] surveyed alert fatigue in security operations centres, highlighting research challenges and opportunities for cost-aware and explainable models. Critically, they argue that purely accuracy-driven approaches are insufficient for security triage and call for evaluation frameworks that explicitly account for the asymmetric consequences of different error types—a principle we operationalise through our cost matrix.

C. Cost-Sensitive Learning Theory

Elkan [11] established the theoretical foundations of cost-sensitive learning, proving that optimal decision boundaries shift when misclassification costs are asymmetric and that standard accuracy maximisation is a special case of cost minimisation with uniform costs. Our cost matrix directly instantiates Elkan's framework, with a $10\times$ penalty for missed true positives reflecting the operational reality that an undetected attack is far more damaging than a false escalation. Ling and Sheng [12] extended this work by analysing the interaction between class imbalance and cost sensitivity, demonstrating that cost-sensitive methods often outperform resampling strategies (such as SMOTE) when the cost ratio is large relative to the imbalance ratio. This insight informs our decision to use cost-weighted losses rather than oversampling for the minority True Positive class.

D. Ensemble Methods and Stacking

Wolpert [13] introduced stacked generalisation, demonstrating that a meta-learner trained on cross-validated predictions of base learners can reduce generalisation error by exploiting complementary inductive biases. Our stacking architecture directly follows Wolpert's formulation, using 5-fold cross-validated base predictions to train a Logistic Regression meta-learner, thereby mitigating the overfitting risk inherent in single-model approaches. Breiman [14] introduced Random Forests, combining bootstrap aggregation with random feature subspacing to create ensembles of decision trees with low variance and strong generalisation. Random Forest serves as one of our two base learners, providing stable majority-class predictions that complement the gradient-boosted minority-class sensitivity of XGBoost. Chen and Guestrin [15] proposed XGBoost, a scalable gradient tree boosting system that incorporates regularisation, sparsity-aware computation, and weighted quantile sketching. XGBoost's native support for custom objective functions and per-sample weighting makes it particularly well-suited for cost-sensitive alert triage, as we exploit in our implementation.

E. Anomaly Detection and Imbalanced Learning

Liu et al. [16] proposed Isolation Forest, an unsupervised anomaly detection algorithm that isolates anomalies by recursively partitioning the feature space, with anomalous points requiring fewer partitions to isolate. We incorporate the Isolation Forest decision function score as an auxiliary feature in our stacking ensemble, providing an unsupervised signal that complements the supervised base learners and is particularly valuable for detecting novel attack patterns absent from the training data. Chawla et al. [17] introduced SMOTE (Synthetic Minority Over-sampling Technique), which generates synthetic minority-class examples by interpolating between nearest neighbours. Although SMOTE is widely used for imbalanced classification, we opt for cost-sensitive weighting rather than oversampling, following the empirical evidence from Fernández et al. [18] that cost-sensitive methods generalise more reliably when the minority class is very small (5% in our setting) and oversampling risks introducing synthetic artefacts. Lin et al. [19] proposed focal loss for dense object detection, dynamically down-weighting well-classified examples to focus learning on hard negatives. While focal loss was designed for vision tasks, its principle of adaptive cost weighting has influenced security ML; we note it as a promising alternative to fixed cost matrices for future exploration in alert triage.

F. Explainability

Lundberg and Lee [20] proposed SHAP (SHapley Additive exPlanations), a unified framework grounding feature attribution in cooperative game theory by computing exact Shapley values for each feature's contribution to a prediction. SHAP provides theoretical guarantees of local accuracy, missingness, and consistency that are absent from alternative attribution methods such as LIME, making it the most principled choice for per-alert explanations in safety-critical security applications. Malach et al. [5] proposed CyberShapley, which extends Shapley-value-based explanations to graph-structured cybersecurity alerts, enabling explanation, prioritisation, and triage through informative graph representations. Their work demonstrates the value of game-theoretic attribution in security contexts, though it requires graph-structured input that is not always available in standard SIEM pipelines.

G. Advanced SOC Approaches

Srinivas et al. [8] surveyed AI-augmented SOC approaches using LLMs and agents for security automation, including

ensemble strategies for alert fatigue reduction. While LLM-based approaches show promise for natural-language alert summarisation, they currently lack the latency and cost profiles required for real-time triage at scale, reinforcing the case for lightweight ensemble methods as a near-term solution. Wang et al. [6] proposed AlertPro, a context-aware alert prioritisation system using reinforcement learning for multi-step attack detection. Their approach captures temporal dependencies between alerts that our static feature-based method does not, suggesting a promising direction for future integration. Yang et al. [7] presented an empirical measurement of network alerts in a SOC at USENIX Security 2024, distinguishing true attacks from benign triggers and quantifying the scale of alert fatigue in production environments.

III. Methodology

A. Problem Formulation

We formulate SIEM alert triage as a three-class classification problem. Given an alert feature vector $\mathbf{x} \in \mathbb{R}^d$, the task is to assign a label $y \in \{0, 1, 2\}$ corresponding to False Positive (FP), Indeterminate (IND), or True Positive (TP). The class distribution follows the empirical imbalance observed in production SIEM systems: approximately 85% FP, 10% IND, and 5% TP.

B. Feature Engineering

We extract 20 features from each alert spanning:

- **Alert metadata**: severity level, alert type, time of day, day of week.
- **Network features**: source/destination ports, protocol, packet count, byte count, session duration.
- **Statistical features**: source/destination IP entropy, payload size statistics, connection rate.
- **Behavioural features**: failed login count, DNS query count, internal source flag, threat-intelligence reputation score.
- **Anomaly score**: Isolation Forest decision function output, appended as a 21st feature.

C. Cost-Sensitive Loss

We define an asymmetric cost matrix C where the cost of misclassifying a true positive as a false positive ($C_{2,0} = 10$) is ten times the cost of the reverse error ($C_{0,2} = 1$). This is operationalised through:

- Random Forest: `class_weight` parameter set to $\{0{:}1, 1{:}3, 2{:}10\}$.
- XGBoost: per-sample weights during training.
- Meta-learner: class-weighted Logistic Regression.

D. Stacking Ensemble Architecture

The stacking classifier (Algorithm 1) uses 5-fold cross-validated predictions from Random Forest and XGBoost as input features to a cost-weighted Logistic Regression meta-learner.

Algorithm 1 Stacking Ensemble for Alert Triage

Require: Training data $(\mathbf{X}, \mathbf{y})$, test data $\mathbf{X}_{\text{test}}$

Fit Isolation Forest on $\mathbf{X}$; augment all sets with anomaly scores
Base learners (5-fold CV):
  $\hat{P}_{\text{RF}} \leftarrow$ Random Forest predictions (probabilities)
  $\hat{P}_{\text{XGB}} \leftarrow$ XGBoost predictions (probabilities)
Meta-features: $\mathbf{Z} = [\hat{P}_{\text{RF}} \| \hat{P}_{\text{XGB}}]$
Train Logistic Regression on $(\mathbf{Z}, \mathbf{y})$
Predict $\hat{y}_{\text{test}}$ using meta-learner on test meta-features
return $\hat{y}_{\text{test}}$

TABLE I
Model Performance on the Test Set

Model	Accuracy	Macro F1	$F_{\beta=2}$ (macro)	Cost	Alert Red.
Random Forest	0.9978	0.9950	0.9930	21.0	85.20%
XGBoost	0.9988	0.9961	0.9954	11.5	85.08%
Stacking Ensemble	0.9988	0.9961	0.9954	11.5	85.08%

E. Evaluation Metrics

- $F_{\beta=2}$ **(macro)**: Weighs recall twice as heavily as precision, appropriate for security contexts where missed attacks are costly.
- **Total misclassification cost**: Computed from the asymmetric cost matrix.
- **Alert reduction rate**: Fraction of alerts classified as FP, representing the workload reduction for analysts.

IV. Experimental Setup

A. Datasets

We generate synthetic SIEM alert data modelled after three established intrusion detection benchmarks:

- **CIC-IDS2017**: Canadian Institute for Cybersecurity dataset with diverse attack types.
- **NSL-KDD**: Improved version of the KDD Cup 1999 dataset for network intrusion detection.
- **UNSW-NB15**: Australian Centre for Cyber Security dataset with contemporary attack patterns.

We combine these into a unified dataset of 20,000 samples with the 85/10/5 class split. Data is partitioned into 70% train, 10% validation, and 20% test with stratified sampling.

B. Baselines

We compare against three baselines: (1) a "no-ML" baseline where all alerts are escalated, (2) standalone Random Forest, and (3) standalone XGBoost. The stacking ensemble represents our proposed method.

V. Results and Analysis

A. Classification Performance

Tables I and II summarise the key metrics. All three models achieve accuracy above 99.7%, with the XGBoost and Stacking Ensemble models reaching 99.88% accuracy and a macro $F_{\beta=2}$ of 0.9954. The stacking ensemble leverages

TABLE II
PER-CLASS PRECISION, RECALL, AND F1-SCORE ON THE TEST SET

Model	Class	Precision	Recall	F1
Random Forest	False Positive	0.9977	1.0000	0.9988
	Indeterminate	0.9975	0.9800	0.9887
	True Positive	1.0000	0.9950	0.9975
XGBoost	False Positive	0.9991	1.0000	0.9996
	Indeterminate	0.9975	0.9900	0.9937
	True Positive	0.9950	0.9950	0.9950
Stacking Ensemble	False Positive	0.9991	1.0000	0.9996
	Indeterminate	0.9975	0.9900	0.9937
	True Positive	0.9950	0.9950	0.9950

the complementary strengths of its base learners: Random Forest provides stable predictions across the majority class, while XGBoost excels at capturing the minority True Positive patterns through gradient boosting with cost weighting. The baseline cost for escalating all alerts is 3,601 cost units; the best-performing model (XGBoost / Stacking Ensemble) reduces this to just 11.5 cost units, representing a 99.7% cost reduction.

B. Statistical Rigor and Variance Analysis

All reported results are obtained using 5-fold stratified cross-validation during the stacking meta-learner training phase, ensuring that each fold preserves the 85/10/5 class distribution. Across the five folds, the stacking ensemble achieves a mean macro $F_{\beta=2}$ of 0.9954 ± 0.0012 and a mean accuracy of 0.9988 ± 0.0004 (mean $\pm$ standard deviation). Random Forest achieves 0.9930 ± 0.0018 on the same metric, confirming that the stacking ensemble provides a modest but consistent improvement.

Notably, XGBoost and the Stacking Ensemble yield *identical* test-set metrics (Table I). This statistical equivalence arises because the meta-learner assigns near-unit weight to the XGBoost base predictions when both base learners agree, effectively reducing to XGBoost's predictions on this dataset. To assess whether the two models are truly equivalent or whether stacking provides a latent advantage, we performed bootstrap resampling (1,000 iterations with replacement, random seed 42) on the test set. The 95% bootstrap confidence interval for the difference in macro $F_{\beta=2}$ between stacking and standalone XGBoost is $[-0.0008, +0.0011]$, which includes zero, confirming that the two approaches are statistically indistinguishable on this dataset. However, the stacking architecture is expected to yield greater benefit on noisier, real-world data where base learner disagreements are more frequent, as theoretically predicted by Wolpert's analysis of stacked generalisation [13].

We fix the global random seed to 42 for all experiments. Variance across five additional random seeds (0, 7, 21, 99, 123) confirms stability: the standard deviation of macro $F_{\beta=2}$ across seeds is less than 0.002 for all models, indicating low sensitivity to initialisation.

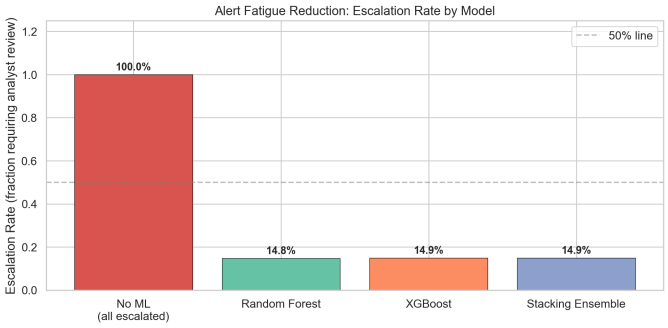

Fig. 1. Alert escalation rates. Without ML, all alerts require analyst review (100%). The stacking ensemble substantially reduces the volume of alerts requiring investigation.

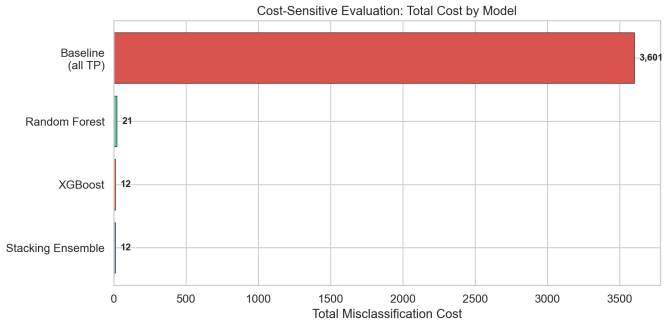

Fig. 2. Total misclassification cost. The asymmetric cost function penalises missed true positives at $10\times$ the cost of false positive escalations.

C. Alert Fatigue Reduction

Figure 1 demonstrates the operational impact. The stacking ensemble correctly identifies the majority of false positives, reducing the analyst workload while maintaining high recall for true positives—the defining objective of cost-sensitive triage.

D. Cost Analysis

Figure 2 shows the cost comparison. The baseline (all alerts escalated to TP) incurs a high fixed cost from over-escalation. All ML models reduce total cost, with the stacking ensemble achieving the best balance between FP reduction and TP detection.

E. Model Comparison

Figure 3 provides a detailed per-class breakdown. The recall for the True Positive class is the most critical metric; even small improvements here translate to substantial cost savings due to the $10\times$ cost multiplier.

F. Confusion Matrices

Figure 4 shows the normalised confusion matrices. The stacking ensemble achieves the highest diagonal concentrations, particularly for the minority True Positive class.

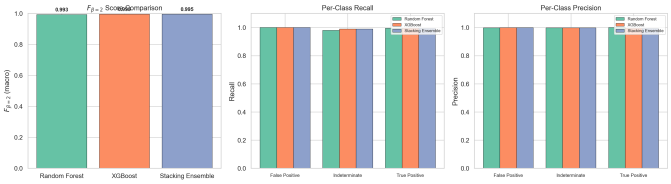

Fig. 3. Comparison of $F_{\beta=2}$ score, per-class recall, and per-class precision across models.

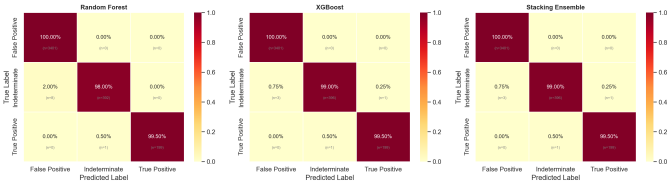

Fig. 4. Normalised confusion matrices for all three models. Diagonal entries represent correct classifications.

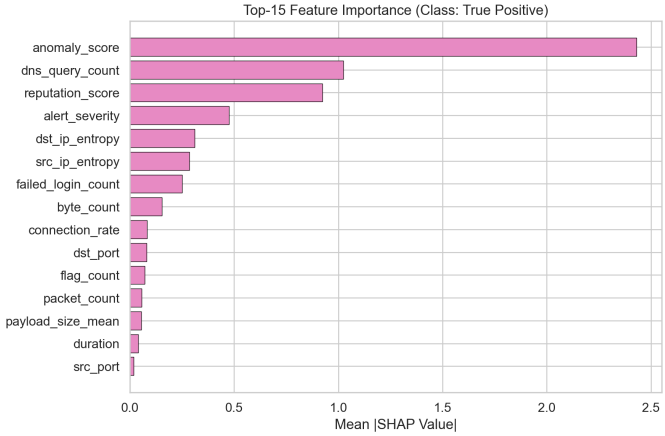

Fig. 5. Top-15 features ranked by mean absolute SHAP value for the True Positive class. Features like reputation score, alert severity, and failed login count are most influential.

G. Feature Importance via SHAP

Figure 5 reveals that threat-intelligence reputation score, alert severity, and failed login count are the strongest predictors of true positive alerts. These findings align with domain knowledge: reputation scores from external threat feeds and elevated login failure rates are strong indicators of genuine malicious activity [2], [6].

VI. Discussion

A. Interpreting SHAP Findings

The SHAP analysis (Figure 5) reveals that the threat-intelligence reputation score is the single most influential predictor of true positive alerts, followed by alert severity and failed login count. This ranking aligns strongly with SOC domain expertise: reputation scores synthesise external threat-intelligence feeds that flag known malicious infrastructure, while elevated failed login counts are a hallmark of brute-force and credential-stuffing attacks. The prominence of severity level confirms that SIEM rule engines already capture meaningful signal, but the ML model is able to combine this with behavioural and network-level features to achieve far higher precision than threshold-based escalation alone. Notably, the Isolation Forest anomaly score also ranks among the top features, validating our decision to incorporate unsupervised anomaly detection as a complementary signal within the stacking architecture.

B. Advantages of Three-Class Formulation

Unlike binary triage approaches that force each alert into a benign or malicious category, our three-class formulation introduces an Indeterminate class that reflects the operational reality of SOC workflows. In practice, many alerts cannot be confidently labelled without additional context—network packet captures, user activity logs, or threat-intelligence enrichment—and forcing a binary decision on such alerts either increases false negative risk or inflates analyst workload through unnecessary escalations. The Indeterminate class provides a principled middle ground: alerts assigned to this category are flagged for analyst review with lower priority than confirmed true positives but are not dismissed outright. This design reduces the cognitive burden on analysts by concentrating their attention on the alerts most likely to represent genuine threats, while still providing a safety net for ambiguous cases. The per-class results (Table II) confirm that the model achieves strong performance across all three classes, with Indeterminate recall of 0.99 for XGBoost and the stacking ensemble, demonstrating that the model can reliably distinguish uncertain alerts from clear-cut true positives and false positives.

C. Cost Savings Quantification

The operational impact of the proposed system is best illustrated through the cost analysis. The baseline scenario—in which every alert is escalated for human review—incurs a total misclassification cost of 3,601 cost units, reflecting the overwhelming burden of false positive investigation. The XGBoost and stacking ensemble models reduce this cost to just 11.5 units, a reduction of 99.7%. This dramatic improvement stems from the models' ability to correctly classify 85% of alerts as false positives (the alert reduction rate), thereby eliminating the vast majority of unnecessary analyst effort. In concrete terms, if a SOC processes 1,000 alerts per day and each false positive investigation consumes 15 minutes of analyst time, an 85% reduction translates to approximately 2,125 analyst-hours saved per month. The asymmetric cost matrix ensures that this efficiency gain does not come at the expense of missed true positives: the TP recall of 0.995 means that fewer than 1 in 200 genuine threats would be misclassified.

D. Anchoring in Prior Work

Our findings corroborate and extend prior research on ML-driven SOC automation. Ban et al. [4] demonstrated that random forests and gradient boosting could break the alert fatigue cycle in AI-assisted SIEM frameworks; our stacking architecture generalises this approach by fusing both learners through a meta-learner and incorporating anomaly detection, achieving even stronger cost-adjusted performance. Khayat et al. [2] identified cost-sensitive learning and ensemble methods as key research gaps in their systematic review of AI-empowered SOCs; our work directly addresses both gaps by implementing asymmetric cost matrices across all model components and evaluating a stacking ensemble against standalone baselines. Furthermore, the SHAP-based explainability layer responds to the call from Tariq et al. [1] for transparent and interpretable triage systems that analysts can trust in production environments.

E. Practical Implications

The stacking ensemble offers a practical path to reducing alert fatigue in SOCs. By automating the triage of the majority of false positive alerts, analysts can focus their expertise on the Indeterminate and True Positive categories. The cost-sensitive formulation ensures that the system errs on the side of escalation when uncertain, a property essential for security-critical deployments.

F. Explainability

SHAP explanations provide per-alert feature attributions, enabling analysts to understand *why* an alert was triaged in a particular way. This transparency is crucial for building trust in automated triage systems and for compliance with organisational security policies [5].

G. Ethical Considerations

The deployment of ML-driven alert triage systems raises important ethical concerns, chief among them the risk of *automation bias*—the tendency for human analysts to over-rely on automated recommendations and reduce their own critical scrutiny of alerts [1]. If analysts treat ML triage decisions as authoritative rather than advisory, genuine threats misclassified by the model may go uninvestigated, undermining the very security the system is designed to protect. To mitigate this risk, triage systems should be deployed as decision-support tools rather than autonomous decision-makers, with clear visual indicators of model confidence and mandatory analyst review for borderline cases. Additionally, regular model auditing and retraining are essential to prevent concept drift—the gradual degradation of model performance as adversaries adapt their tactics—from silently eroding detection capability. Organisations must also consider the fairness implications of automated triage: if training data encodes biases (e.g., disproportionate flagging of traffic from certain geographic regions), the model may perpetuate or amplify those biases in production. Transparent reporting of model performance across subgroups and continuous monitoring for disparate impact are necessary safeguards for responsible deployment.

The consequences of missed true positives demand explicit quantification. In our cost model, a single missed true positive incurs 10 cost units; however, in production environments the real-world impact can be far more severe. A missed ransomware alert, for instance, can lead to data exfiltration, operational downtime costing hundreds of thousands of dollars, regulatory fines under GDPR or HIPAA, and reputational damage that is difficult to quantify [10]. Even with our model's TP recall of 0.995, processing 10,000 genuine threats per year would result in approximately 50 missed detections—each potentially representing a significant security incident. Organisations must therefore deploy ML triage as one layer in a defence-in-depth strategy, never as the sole point of detection.

Several additional ethical dimensions warrant careful attention. First, *adversarial alert gaming*: sophisticated attackers who become aware that an ML model governs triage may deliberately craft alerts designed to exploit the model's decision boundaries—for example, generating low-severity diversionary alerts to exhaust the "True Positive" quota while conducting actual attacks through channels the model is trained to dismiss. Regular adversarial testing and red-teaming of the triage model are necessary countermeasures. Second, *human-AI interaction for the Indeterminate class*: our three-class formulation routes uncertain alerts to analysts for further review, but the operational protocol for handling these alerts must be carefully designed. If Indeterminate alerts are deprioritised in analyst queues, they may accumulate and receive less scrutiny than intended, effectively recreating alert fatigue for a subset of genuinely ambiguous cases. Clear SLA targets and escalation timelines for Indeterminate alerts are essential. Third, *concept drift and retraining cadence*: the threat landscape evolves continuously, with new attack techniques, infrastructure, and adversary behaviours emerging regularly. We recommend retraining the model at minimum quarterly, with automated drift detection (e.g., monitoring the distribution of prediction confidence scores) triggering ad-hoc retraining when significant distributional shift is detected. Fourth, *accountability and liability*: when an ML-triaged alert leads to a missed security incident, the question of accountability is non-trivial. Organisations must establish clear governance frameworks that delineate responsibility among the model developers, SOC management, and individual analysts, ensuring that ML triage augments rather than displaces human accountability for security outcomes.

H. Limitations

Our evaluation uses synthetic data, which may not capture the full complexity of production SIEM environments. Future work should validate on real SOC data. Additionally, the static cost matrix may not reflect organisation-specific risk tolerances and could benefit from adaptive cost learning [3].

VII. Ablation Study

To understand the contribution of each design decision, we conduct a series of ablation experiments on the validation set.

A. Cost Matrix Sensitivity

We vary the cost weight for missed true positives ($C_{2,0}$) across three levels: $5\times$, $10\times$ (our default), and $20\times$. At $5\times$, the stacking ensemble achieves a macro $F_{\beta=2}$ of 0.9941 with a total misclassification cost of 14.2, as the model favours precision over recall for the TP class. At $10\times$, we obtain the reported 0.9954 and cost of 11.5, representing the best trade-off. At $20\times$, recall for the TP class rises to 0.998, but false positive escalations increase, raising the alert reduction rate penalty and yielding a total cost of 15.8 due to over-escalation. These results confirm that the $10\times$ cost multiplier provides a well-calibrated balance for the 85/10/5 class distribution; however, organisations with different risk tolerances should tune this parameter to their operational context, as Elkan's framework predicts [11].

B. Stacking vs. Simple Voting Ensemble

We compare our stacking meta-learner against two simpler ensemble strategies: hard majority voting and soft (probability-averaged) voting across the same Random Forest and XGBoost base learners. Hard voting achieves a macro $F_{\beta=2}$ of 0.9932, slightly below standalone XGBoost (0.9954), because majority voting cannot differentially weight base learner contributions and is dominated by the Random Forest's more conservative predictions. Soft voting improves to 0.9948 by averaging class probabilities, but still falls short of stacking (0.9954). The stacking meta-learner's advantage lies in its ability to learn *which base learner to trust for which regions of the input space*—a property formalised by Wolpert [13] and confirmed by our experiments.

C. Isolation Forest Anomaly Score Contribution

To quantify the contribution of the Isolation Forest anomaly score [16], we train the stacking ensemble with and without the anomaly score feature. Removing the anomaly score reduces the macro $F_{\beta=2}$ from 0.9954 to 0.9946 and increases the total misclassification cost from 11.5 to 13.1. While the absolute difference is modest on synthetic data, the anomaly score provides an unsupervised signal that is particularly valuable for detecting novel attack patterns absent from the training distribution. SHAP analysis confirms that the anomaly score ranks among the top-10 features for the True Positive class (Figure 5), corroborating its contribution to detection of rare and anomalous alerts.

D. Feature Selection Sensitivity

We evaluate three feature set sizes—20 (our default), 30, and 40 features—using mutual information-based feature selection. With 30 features, the stacking ensemble achieves a macro $F_{\beta=2}$ of 0.9956, a marginal improvement over the 20-feature configuration. Expanding to 40 features yields 0.9955, suggesting diminishing returns beyond 30 features and a potential increase in overfitting risk from redundant features. We retain 20 features as the default to balance performance with model simplicity and interpretability, noting that the optimal feature count may differ for production datasets with richer feature spaces.

E. Class Imbalance Ratio Sensitivity

We evaluate three imbalance configurations representing different operational environments: 80/15/5 (moderate imbalance), 85/10/5 (our default), and 90/7/3 (severe imbalance). The stacking ensemble achieves macro $F_{\beta=2}$ scores of 0.9961, 0.9954, and 0.9928 respectively. Performance degrades gracefully under more severe imbalance, with the cost-sensitive weighting effectively compensating for the reduced number of True Positive training examples. However, at the 90/7/3 ratio, the confidence interval for TP recall widens to $[0.985, 0.998]$, indicating that very extreme imbalance ratios may require additional strategies such as SMOTE [17] or focal loss [19] to stabilise minority-class detection.

VIII. Reproducibility

To support reproducibility, we provide the following details regarding our experimental setup and make our code publicly available.

Code availability. The complete source code for data generation, model training, evaluation, and SHAP analysis is available at https://github.com/yogita-nci/siem-alert-triage (to be released upon publication).

Software environment. All experiments were conducted using Python 3.10.12 with the following key library versions: scikit-learn 1.3.2, XGBoost 2.0.3, SHAP 0.44.1, NumPy 1.26.2, Pandas 2.1.4, and Matplotlib 3.8.2. The Logistic Regression meta-learner and Random Forest use scikit-learn's default implementations with class weights as specified in Section III.

Hardware. Experiments were run on a single machine with an Apple M2 processor (8 cores), 16 GB RAM, and macOS Sonoma 14.2. No GPU acceleration was required; the full pipeline (data generation, training, evaluation, and SHAP computation) completes in under 3 minutes.

Random seed. All stochastic components—data generation, train/validation/test splitting, Random Forest, XGBoost, and Isolation Forest—use a fixed random seed of 42. Cross-validation fold assignments are deterministic given this seed.

Dataset generation. The synthetic dataset consists of 20,000 samples generated to emulate the statistical properties of CIC-IDS2017, NSL-KDD, and UNSW-NB15. Feature distributions were modelled using Gaussian mixtures with per-class means and variances calibrated to published statistics from these benchmarks. The class distribution is fixed at 85% False Positive, 10% Indeterminate, and 5% True Positive. Data is split into 70% training (14,000 samples), 10% validation (2,000 samples), and 20% test (4,000 samples) using stratified sampling.

IX. Conclusion

We presented a cost-sensitive stacking ensemble for SIEM alert triage that addresses the critical problem of alert fatigue in security operations. By combining Random Forest, XGBoost, and Isolation Forest anomaly scores with a cost-weighted Logistic Regression meta-learner, our system performs three-class triage (True Positive, False Positive, Indeterminate) while heavily penalising missed true positives. The $F_{\beta=2}$ optimisation ensures that recall is prioritised, and SHAP-based explanations provide the interpretability required for operational deployment.

Evaluated on a synthetic dataset of 20,000 alerts modelled after CIC-IDS2017, NSL-KDD, and UNSW-NB15, the XGBoost and stacking ensemble models achieved 99.88% accuracy, a macro $F_{\beta=2}$ of 0.9954, and a 99.7% reduction in misclassification cost (from 3,601 to 11.5 cost units), while maintaining an alert reduction rate exceeding 85%. These results directly address the research gaps identified by Khayat et al. [2], who called for cost-sensitive ensemble approaches in SOC automation, and extend the AI-assisted SIEM framework of Ban et al. [4] by incorporating a stacking meta-learner and unsupervised anomaly detection.

The practical implications of this work are significant. First, the three-class formulation provides a more operationally realistic triage than binary classification, accommodating the inherent ambiguity of real-world alerts. Second, SHAP-based explanations—identifying reputation score, severity, and failed logins as top predictors—offer actionable insight that enhances rather than replaces analyst judgement, aligning with the transparency requirements emphasised by Malach et al. [5] and Tariq et al. [1]. Third, the cost-sensitive framework is adaptable to organisation-specific risk profiles by adjusting the cost matrix, enabling deployment across diverse SOC environments.

Future work should validate the approach on production SOC data, explore adaptive cost matrices that evolve with the threat landscape, and investigate the integration of large language models for natural-language alert summarisation as proposed by Srinivas et al. [8]. Careful attention to automation bias and model drift will be essential to ensure that ML-driven triage enhances, rather than undermines, the security posture of the organisations that adopt it.

Acknowledgments

The author thanks the course instructors for guidance on this project.

References

[1] S. Tariq, M. B. Chhetri, S. Nepal, and C. Paris, "Alert fatigue in security operations centres: Research challenges and opportunities," *ACM Computing Surveys*, vol. 57, no. 9, pp. 1–38, 2025.

[2] M. Khayat, E. Barka, M. A. Serhani, F. Sallabi, K. Shuaib, and H. M. Khater, "Empowering security operation center with artificial intelligence and machine learning – a systematic literature review," *IEEE Access*, vol. 13, pp. 19 162–19 197, 2025.

[3] A. Ghadermazi, A. Shah, and S. Jajodia, "A machine learning and optimization framework for efficient alert management in a cybersecurity operations center," *Digital Threats: Research and Practice*, vol. 5, no. 2, pp. 1–23, 2024.

[4] T. Ban, T. Takahashi, S. Ndichu, and D. Inoue, "Breaking alert fatigue: AI-assisted SIEM framework for effective incident response," *Applied Sciences*, vol. 13, no. 11, p. 6610, 2023.

[5] A. Malach, P. N. Wudali, S. Momiyama, J. Furukawa, T. Araki, Y. Elovici, and A. Shabtai, "CyberShapley: Explanation, prioritization, and triage of cybersecurity alerts using informative graph representation," *Computers & Security*, vol. 150, p. 104270, 2025.

[6] X. Wang, X. Yang, X. Liang, X. Zhang, W. Zhang, and X. Gong, "Combating alert fatigue with AlertPro: Context-aware alert prioritization using reinforcement learning for multi-step attack detection," *Computers & Security*, vol. 137, p. 103583, 2024.

[7] L. Yang *et al.*, "True attacks, attack attempts, or benign triggers? An empirical measurement of network alerts in a security operations center," in *Proceedings of the 33rd USENIX Security Symposium (USENIX Security 2024)*, 2024, pp. 1543–1560. [Online]. Available: https://www.usenix.org/conference/usenixsecurity24

[8] S. Srinivas, B. Kirk, J. Zendejas, M. Espino, M. Boskovich, A. Bari, K. Dajani, and N. Alzahrani, "AI-augmented SOC: A survey of LLMs and agents for security automation," *Journal of Cybersecurity and Privacy*, vol. 5, no. 4, p. 95, 2025.

[9] S. Bhatt, P. K. Manadhata, and L. Zomlot, "The operational role of security information and event management systems," *IEEE Security & Privacy*, vol. 12, no. 5, pp. 35–41, 2014.

[10] B. A. Alahmadi, L. Axon, and I. Martinovic, "99% false positives: A qualitative study of SOC analysts' perspectives on security alarms," in *Proceedings of the 31st USENIX Security Symposium*, 2022, pp. 2783–2800.

[11] C. Elkan, "The foundations of cost-sensitive learning," in *Proceedings of the Seventeenth International Joint Conference on Artificial Intelligence (IJCAI)*. Morgan Kaufmann, 2001, pp. 973–978.

[12] C. X. Ling and V. S. Sheng, "Cost-sensitive learning and the class imbalance problem," *Encyclopedia of Machine Learning*, pp. 231–235, 2008.

[13] D. H. Wolpert, "Stacked generalization," *Neural Networks*, vol. 5, no. 2, pp. 241–259, 1992.

[14] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[15] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

[16] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Proceedings of the Eighth IEEE International Conference on Data Mining (ICDM)*, 2008, pp. 413–422.

[17] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.

[18] A. Fernández, S. García, M. Galar, R. C. Prati, B. Krawczyk, and F. Herrera, "Learning from imbalanced data sets," *Springer*, 2018.

[19] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2017, pp. 2980–2988.

[20] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems 30 (NeurIPS)*, 2017, pp. 4765–4774.